

Mapping (implementing schema)

Database : collection of tables (will be refined later)

Server : A collection of Databases

Table / Entity : collection of records.

Attribute / Column / Field : Characteristic of an entity.

Row / Record / Tuple : the specific characteristic of one entity.

In schema : Primary key => underline by solid line , foreign key => underline by dotted line , and an arrow pointing from the FK to the PK

Tables that has the Primary Key => Parent Table

Table that has the Foreign Key => Child Table

Foreign Key: Can be null , can be repeated (NOT Unique) , some tables have FK and others don't have, the value of FK must be a real PK value, a table can contain multiple foreign keys.

Note : **by default (can be changed)** , we cannot delete the parent that has childs or edit his id , ex: if we want to delete Department 30 , then we must first delete any employee in department 30 to be able to delete this department , same thing if we want to change the ID manually , we must first see if it has childs.

Each column in the database has a domain : allowed range of values that can be held by a specific column , first thing : the datatype specifies a range of values (ex: tinyint from 0 to 255) , second thing : we can add our constraints on the column , ex: age must be less than 60 (even if age data type is tinyint and has max = 255). This will be discussed later in advanced SQL.

A requirement document can produce many ERDs , but one ERD produces only one right schema.

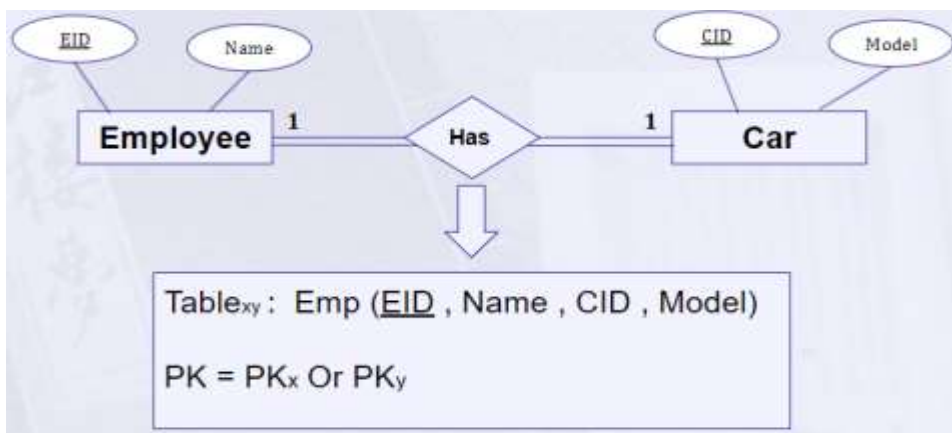
Entity in ERD is NOT a Table DB.

Relation in Mapping schema is a table in DB

Steps for mapping :

- 0- Mapping 1-1 relationship mandatory from 2 sides (case from step 3) .
- 1- Mapping of regular (Strong) entity types.
- 2- Mapping of weak entity types.
- 3- Mapping of binary 1-1 relationship types.
- 4- Mapping of binary 1-N relationship types.
- 5- Mapping of binary M-N relationship types.
- 6- Mapping of N-ary relationship types.
- 7- Mapping of Unary relationship.

Step 0 : binary 1-1 relationship types Both sides are mandatory : are put in one table and the primary key is ANY one of the two primary keys (use the most significant one)



Step 1 : strong entity, with (simple , composite , multi-valued , complex) attributes

(a) Employee entity type with simple attributes:



(b) Employee relation:

Employee		
<u>ID</u>	Name	Address

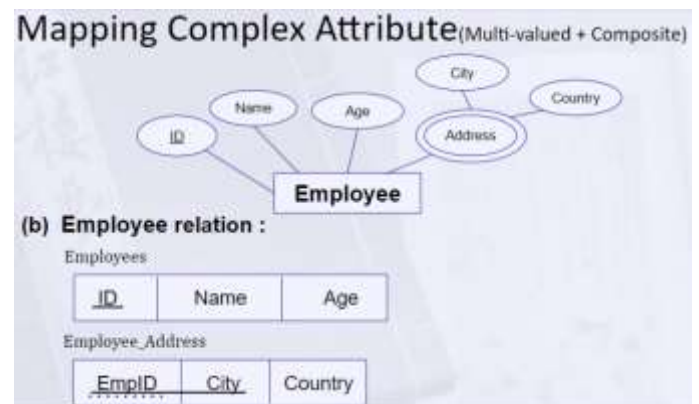
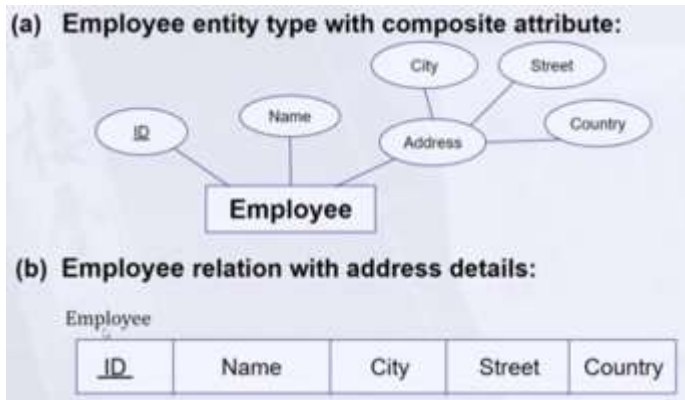
(a) Employee entity type with Multi-Valued attribute:



(b) Employee relation :

Employee		
<u>ID</u>	Name	Age

Employee_Skills	
<u>EmpID</u>	Skill

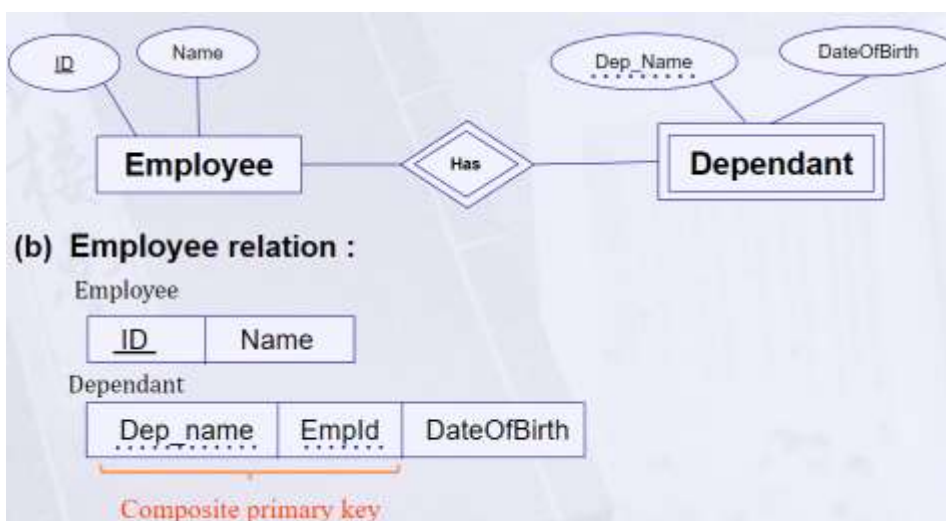


Note :

- In multivalued and complex attributes , we make a new table , and the PK is a composite Pk of the (Foreign Key , attributes) , the combination of the PK must be unique.
- In composite and complex attributes , we put the parts of attribute that were specified , not the actual attribute (This must be specified in the Document after finishing implementation of the database and given to the Database users , ex: address is the concatenation of City and Country)
- Derived or Computed columns are deleted when mapping , they will be computed in the runtime (specify in the Document after finishing implementation of the database that the NetSalary=Salary+Overtime)
- In multi valued attributes , ex: if I know that I have phone1 and phone2 ONLY and we will not add any phones in the future , then it's better to make them as simple attributes for faster reading and querying, otherwise make them as a multi valued attribute for future extensions but this will be slightly slower due to the Joins when querying

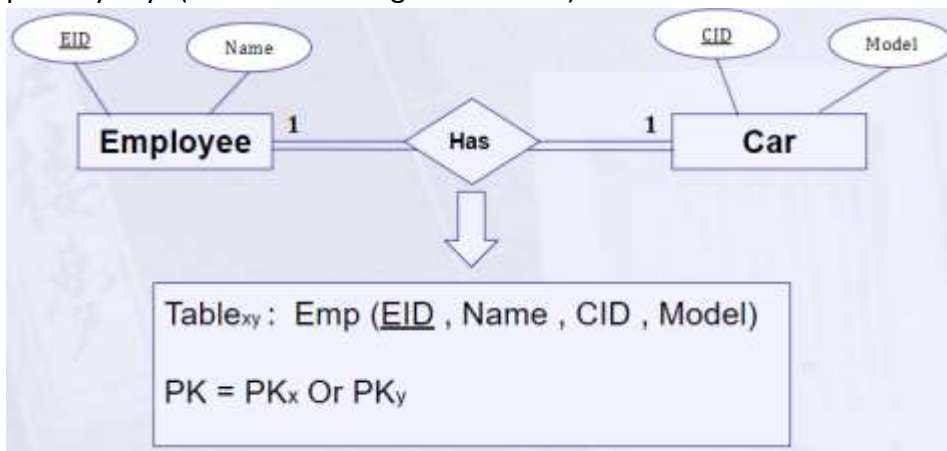
Step 2 : weak entity

- We take the PK of the strong entity as a FK in the weak entity , and in the Weak entity the composite PK is (FK,PartialKey)

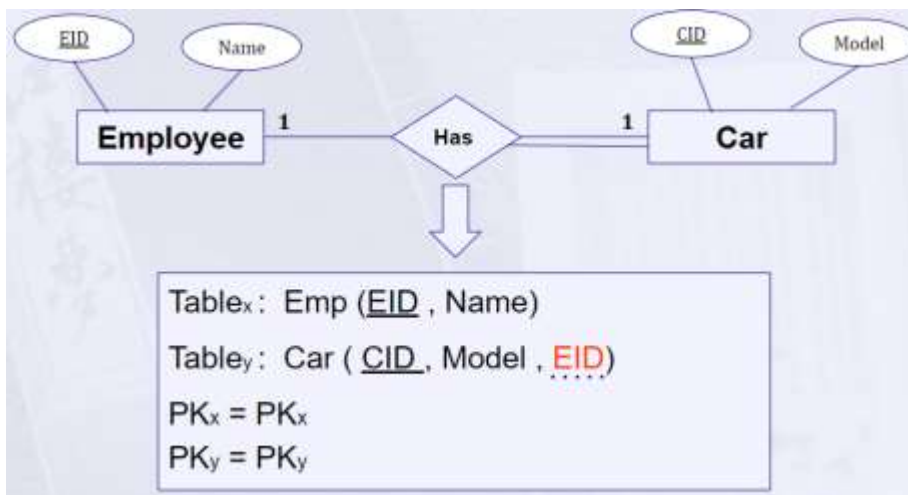


Step 3 : binary 1-1 relationship types , three cases =>

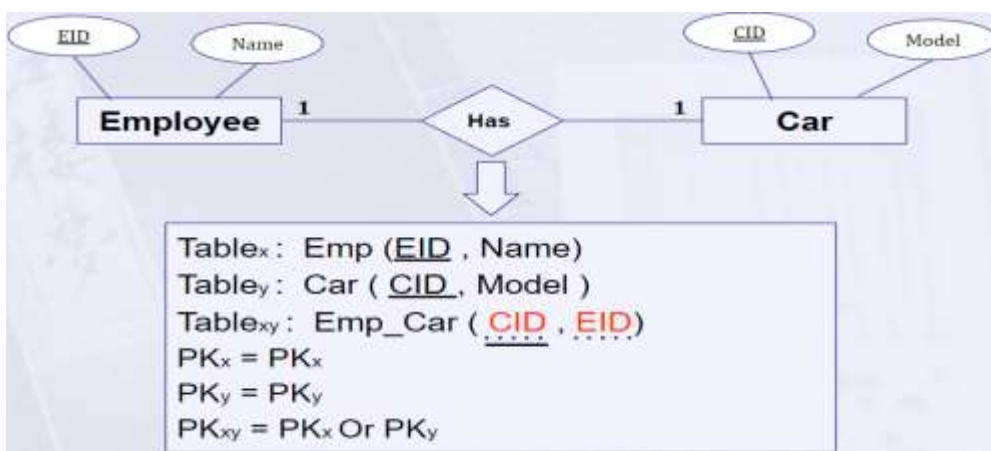
- 1- Both sides are mandatory (step 0) : are put in one table and the primary key is any one of the two primary keys (use the most significant one)



- 2- Side mandatory & side optional : PK of optional as a FK in mandatory side

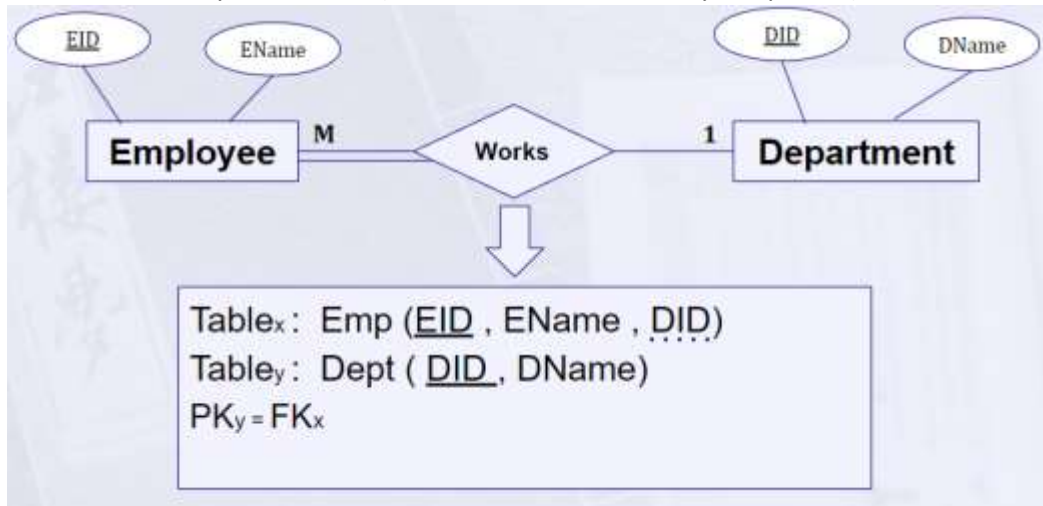


- 3- Both sides are optional : make a new table with composite PK (Two FK) if Unique

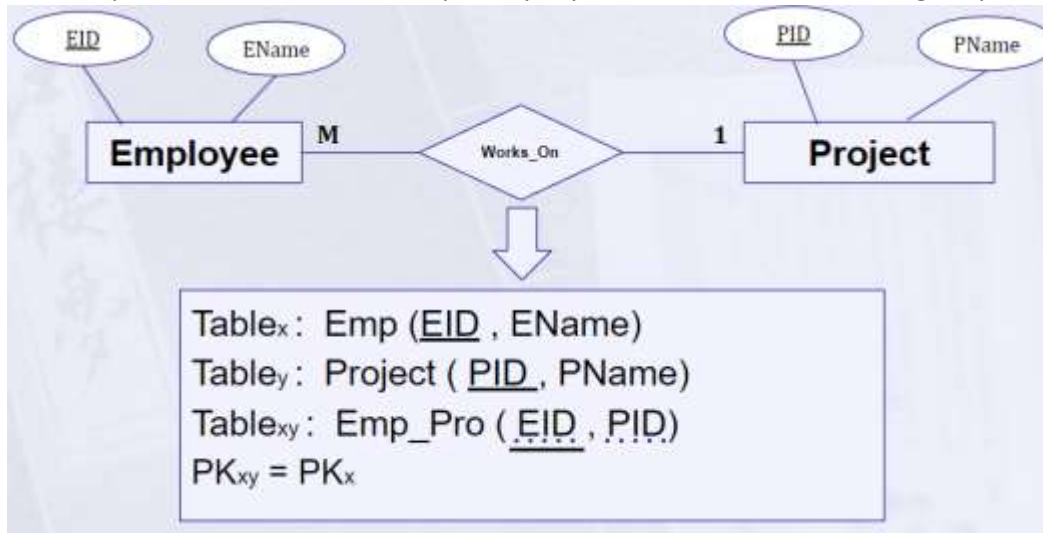


Step 4: Mapping of binary 1-M relationship types, we look at the participation on the MANY side only =>

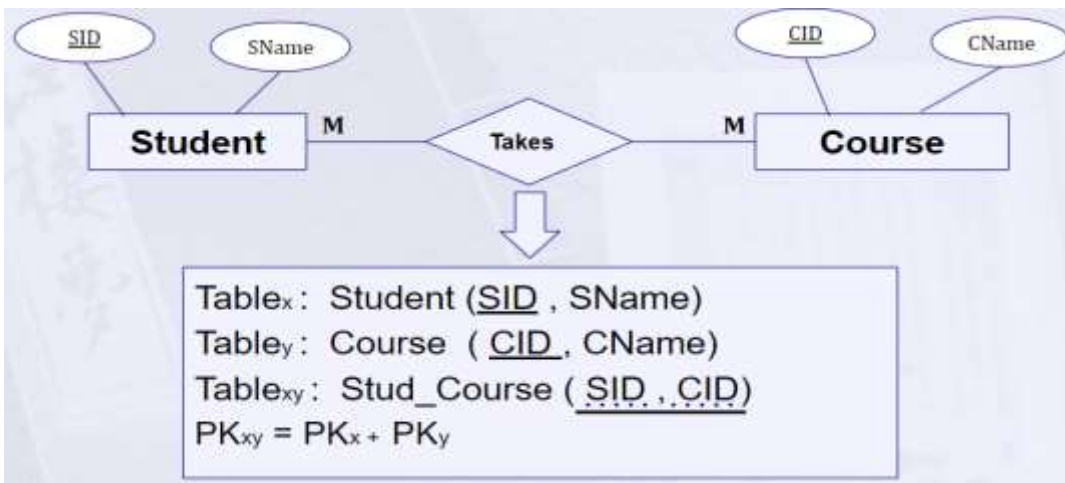
- 1- M-side Mandatory: PK of the 1 (whatever it was mandatory or optional) as a FK in the many side



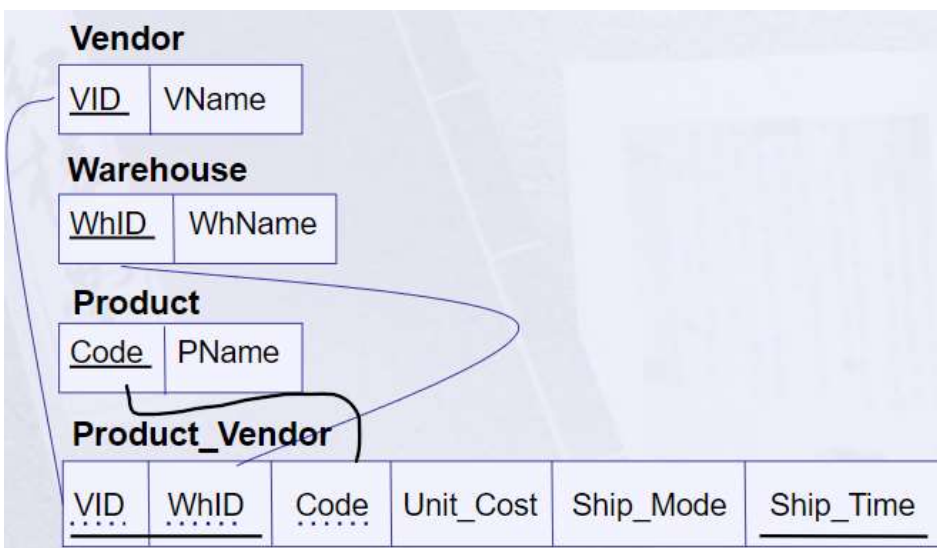
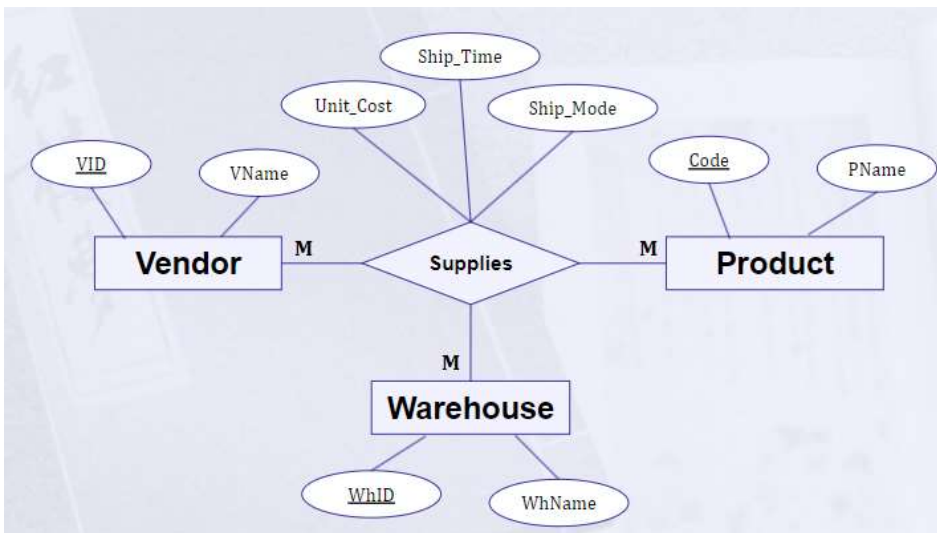
- 2- M-side optional : New table and the primary key in the new table is the foreign key from the MANY side



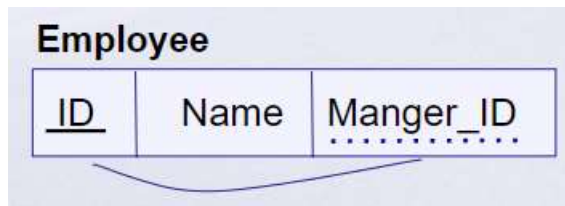
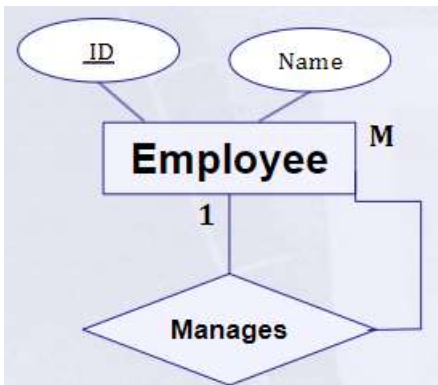
Step 5 : Mapping of binary M-M relationship types (only one way for mapping) => make a new table and the primary key is then a composite primary key (2 FK)



Step 6: Mapping of N-ary relationship types => only one way of mapping (N PK as N FK in the new table and trying to find a combination that will be unique to be a composite primary key).



Step 7 : mapping of unary (self or recursive) relationship => only one case to be solved



Summary:

1 - 1 => One Table , Choose PK of any entity to this table.

1 - 1 => 2 Tables , Take PK Of Optional As FK In Mandatory Table.

1 - 1 => 3 Tables , Choose PK of any entity to this table.

1 - M => 2 Tables , Take PK Of One As FK in Many

1 - M => 3 Tables , PK of Third Table is PK of Many

M - M => 3 Tables , PK of Third Table is Composite PK (PK1 + PK2)

Ternary => Table for Each Entity

+ Table for Relation Contains PKs of each table as FK
(PK is Composite PK).

Notes:

- 1- In multi valued attribute , the multi valued attribute and the foreign key are taken as a COMPOSITE PRIMARY KEY
- 2- Derived attribute isn't stored in the DB , so it's not in the mapping schema.
- 3- When choosing a composite primary key , try to find a combination that will be unique , if not found then insert a new column to be unique and it will be the primary key.
- 4- Attribute on relation goes with the foreign key.